

CashTrack

A Unified Task and Financial Management Dashboard for Freelance Agencies

Final Project Report

Submitted by:

Syed Muhammad Umer Ali

Huda Kanwal

Iqra University

Web Programming Language

Instructor: Faraz Abdul Basit

Abstract

Most small freelance teams end up running their business on two tools that were never meant to talk to each other. Task boards like Trello keep track of what needs to get done, and a separate spreadsheet keeps track of who gets paid for it. The two live in different tabs, get updated by different people, and drift apart within a few weeks. We built CashTrack to close that gap by making the task list and the financial ledger the same underlying data instead of two things someone has to reconcile by hand.

CashTrack is a full-stack web application built with a React.js frontend, a custom REST API written in PHP, and a MySQL database, all developed locally through XAMPP. The application gives agency admins a dashboard to manage clients, assign tasks to team members, and generate invoices, while team members get a simpler view scoped to only the work assigned to them. The part we are most proud of is the automated pipeline that connects the two halves of the app: when an admin marks a task as complete and later marks the related invoice as paid, the system automatically records a transaction and updates that team member's payout history without anyone touching a spreadsheet.

Beyond the functional goal of building something usable, this project was also our first real attempt at connecting a modern JavaScript frontend to a backend we wrote ourselves, rather than relying on a backend-as-a-service platform. That decision shaped a lot of the report that follows, because a good portion of our time went into understanding REST APIs, JSON, PDO prepared statements, and how to keep a database schema clean enough that tasks, invoices, and payouts all reference each other correctly instead of duplicating information. This report walks through the reasoning behind our technology choices, the architecture we ended up with, the database design, the core features, and the mistakes we made and fixed along the way.

1. Introduction & Problem Statement

Almost every small freelance agency we looked at while planning this project runs into the same basic problem. The team needs a place to track what work is being done, and it needs a place to track money — who invoiced what, who got paid, and how much is still owed. In practice, those

two needs get solved with two completely separate tools. A Trello board or something similar handles the tasks. A shared Google Sheet or an Excel file handles the money. Nobody designed it this way on purpose; it is just what happens when a two- or three-person team grows past the point where they can keep everything in their heads.

The problem is that these two systems have no idea the other one exists. A team member finishes a piece of work and moves the card to "Done" on the board. That has zero effect on the spreadsheet. Someone still has to remember to open the spreadsheet, find the right row, and manually add an invoice line for that task. If they forget, or if they get the amount wrong, or if two people update the same cell at the same time, the numbers stop matching reality. We have both seen this happen firsthand in small freelance and part-time work we have done outside of school, and it was frustrating enough that when it came time to pick a final project, we knew we wanted to build something that actually fixed it rather than just another to-do list clone.

Our solution, CashTrack, treats tasks and money as two views into the same set of data instead of two separate systems that someone has to keep in sync by hand. A task in our database is not just a description and a due date — it carries a compensation amount and a status. When that status changes to complete, it becomes eligible to appear on an invoice. When that invoice gets marked as paid, the payout tied to that task updates automatically. Nothing about that flow requires a person to copy a number from one place to another. That is the core idea behind what we kept calling, half-jokingly, our "unified dashboard" philosophy — one dashboard, one underlying dataset, two different lenses depending on whether you are an admin managing the business or a team member checking on your own work.

We also picked this project because it forced us to build a real backend instead of leaning on a hosted database service that handles all the hard parts for you. Plenty of student projects reach for something like Firebase or Supabase and get a working app quickly, but we wanted to actually understand what happens between a button click in React and a row changing in a database. That decision made the project harder and slower at the start, and there were points where we questioned it, but by the end it was the part of the project that taught us the most, and it shows up throughout the rest of this report.

It also matters that this is meant to be a genuinely usable tool and not just a demonstration of a concept. We deliberately avoided building features that would look nice in a five-minute presentation but would not survive a second week of actual use, like a task board with no way to filter it once it has more than a dozen cards, or an invoice screen that only handles one currency and breaks the moment a second client pays in something else. We tried to keep asking ourselves, at every stage, whether a small agency could hand this to a new team member on their first day and have them understand what they were looking at without a walkthrough. That question shaped more of the smaller design decisions in this project than any single technical requirement did.

2. Project Objectives

Functional Objectives

- Provide role-based views so admins and regular team members see different things when they log in, without maintaining two separate applications.
- Support automated invoice generation directly from completed tasks, instead of an admin re-typing task details into a separate invoice form.
- Track payouts to individual team members based on the tasks and invoices tied to their work.
- Support basic currency handling so an agency working with clients in different currencies can still see consistent totals.

Technical & Learning Objectives

- Learn how to structure a React application into components that actually make sense, rather than one giant file that does everything.
- Understand REST APIs well enough to design our own endpoints and reason about request and response shapes, not just consume someone else's API.

- Practice database normalization in MySQL so that clients, tasks, invoices, and payouts reference each other cleanly through foreign keys.
- Learn how to connect a React frontend to a PHP backend securely, including authentication and making sure one user cannot see another user's data just by changing a URL.

We wrote these objectives down together fairly early on, mostly because our first attempt at planning the project was just a list of features with no real structure behind it, and we kept getting distracted building things that looked impressive in a demo but did not actually move us toward a working system. Splitting the objectives into what the app needed to do versus what we personally needed to learn helped us make decisions later on when we were short on time, because we could ask which option actually served one of these goals instead of just picking whichever felt more interesting that week.

3. Technology Stack & Justification

Neither of us had built a full-stack application completely from scratch before this project, so a lot of the stack decisions came down to picking tools where at least one of us had some existing familiarity, balanced against tools that were clearly the standard way of doing things for what we were trying to build.

React.js & Vite

CashTrack has a lot of interconnected views — a task list that needs to reflect changes made from an invoice screen, a dashboard that summarizes numbers pulled from three or four different tables. Trying to manage that with plain HTML and jQuery, which is closer to what we had used in earlier coursework, would have meant manually finding and updating DOM elements every time something changed anywhere in the app. React's component model let us break the interface into pieces — a TaskCard, an InvoiceRow, a PayoutSummary — that each manage their own little piece of state and re-render on their own when the data underneath them changes. We used Vite instead of Create React App mostly because everyone in a project Discord we were part of

recommended it, and once we tried it, the instant reload when saving a file made the trial-and-error part of frontend development a lot less painful than waiting on a slow build every time.

Tailwind CSS

We wanted the dashboard to have a glassmorphism look — translucent cards, soft shadows, blurred backgrounds — and we quickly found that hand-writing CSS for that kind of effect across dozens of components got repetitive fast. Tailwind's utility classes let us apply things like backdrop blur, opacity, and border radius directly in the JSX without constantly switching between a component file and a separate stylesheet. It also meant that when we wanted to tweak a shade of blue across the entire admin dashboard, we were searching and replacing a class name rather than hunting through several CSS files trying to remember which selector controlled which element.

PHP (REST API)

We picked PHP for the backend mainly because one of us had used it briefly in a web development course the semester before, and it pairs naturally with XAMPP, which gave us a local server environment we could set up in an afternoon instead of a full day of configuration. PHP also let us write a genuinely separate backend rather than mixing server logic directly into the frontend, which was important to us given our learning objectives around REST APIs. In our setup, PHP acts as a secure middleman: React never talks to MySQL directly, it only ever talks to a PHP endpoint, and that endpoint decides what is allowed and what gets passed through to the database.

MySQL & phpMyAdmin

A relational database was really the only sensible option once we started mapping out how clients, tasks, invoices, and payouts relate to each other. A task belongs to one client and one assigned team member. An invoice can bundle together several tasks. A payout ties back to both a task and the invoice that covered it. Those are exactly the kinds of relationships a relational database is built to represent through foreign keys, and trying to force that structure into a document-based database would have meant either duplicating data everywhere or writing a lot

of extra application code to keep things consistent. phpMyAdmin gave us a visual way to inspect tables and run quick queries while debugging, which mattered a lot in the early weeks when we were still getting our schema right. One smaller decision worth mentioning here is that we used UUIDs rather than simple auto-incrementing integers for our primary keys, mostly so that IDs referenced in the frontend could not be easily guessed or enumerated by just changing a number in a URL.

XAMPP

XAMPP bundled Apache, MySQL, and PHP together in a way that let us get a local server running without configuring each piece separately, which mattered a lot given how much time we were already spending untangling other parts of the stack. For demonstration purposes, XAMPP was enough to run the whole application locally and show it working end to end, though as we mention later in this report, it is not something we would treat as a long-term solution if this project moved beyond a classroom setting.

4. System Architecture & Data Flow

The overall flow of data through CashTrack follows the same basic pattern for almost every action in the app, whether that is loading the task list or submitting a new invoice. On the frontend, a React component calls the Fetch API to send a request, usually a GET to load data or a POST to create or update something, and that request goes out as JSON. A PHP file sitting on the server receives that request, checks that the user is logged in and allowed to do what they are asking to do, and then talks to MySQL using PDO prepared statements rather than building raw SQL strings by hand. MySQL runs the query and hands rows back to PHP, PHP shapes that into a JSON response, and React receives that response and updates whatever part of the interface depends on it.

Two smaller pieces of that flow gave us more trouble than we expected going in. The first was CORS. Because our React development server and our PHP files were technically running on different ports during development, the browser blocked our very first requests outright and it took us a while, and a fair amount of searching through error messages, to understand that this

was not a bug in our code but a browser security feature that we needed to explicitly allow for by setting the right headers on the PHP side. Once we added the appropriate Access-Control headers to our API responses, the requests started going through, but it was a confusing afternoon before we got there.

The second recurring annoyance was naming conventions. MySQL columns in our schema follow snake_case, which is the normal convention for SQL, things like task_id or due_date. React and JavaScript in general lean toward camelCase, taskId and dueDate. Early on we just left this inconsistent, and it led to bugs where a component expected dueDate and silently got undefined because the API was still sending due_date. We ended up writing a small mapping step inside our PHP response functions that converts snake_case keys to camelCase before the JSON goes out, so that the frontend never has to think about database column naming at all. It was a small fix, but it removed an entire category of bugs we kept running into.

5. Database Design & Schema

Our schema ended up with six main tables, and getting the relationships between them right took more redesigning than any other part of the project.

- profiles — stores every user of the system, both admins and team members, along with a hashed password and a role field that determines what they can see and do.
- clients — stores contact information for the agency's clients, referenced by tasks and invoices.
- tasks — the center of the whole system. Each task links to a client and to the profile of the team member assigned to it, and carries a status, a priority, tags, and a compensation amount.
- invoices — links to a client, tracks a status such as draft, sent, or paid, and stores its line items as a JSON field rather than a separate items table.
- transactions — records income and expenses, linking back to the invoice that generated them.

- payouts — tracks what a specific team member earned from a specific task or invoice, which is what powers the earnings view on the member side of the dashboard.

The relationships between these tables are what let the automated pipeline described later in this report actually work. A task references a `client_id` and an `assigned_to` profile id. An invoice can reference multiple tasks, and once we settled on storing invoice line items as a JSON field, adding or removing an item from a draft invoice stopped requiring a separate insert or delete on another table, which was a real ease-of-implementation win, even though it meant we lost some of the querying flexibility a fully separate items table would have given us. A payout row references both a `task_id` and an `invoice_id`, which is what lets us answer a question like "how much did this person earn from this specific invoice" without scanning through unrelated data.

We went through at least three different versions of this schema before landing here. Our first attempt tried to track payment status directly on the tasks table itself, which seemed simpler at the time, but it fell apart as soon as one invoice needed to cover several tasks at once, because a single paid flag on a task cannot represent "this task is part of an invoice that has been partially paid." Separating payment tracking out into its own transactions and payouts tables, connected through foreign keys back to tasks and invoices, was the fix, and it is the version of the schema described here.

Choosing JSON for the invoice line items instead of a fully separate `invoice_items` table was one of the more debated decisions between the two of us. One of us wanted a proper items table from the start, arguing it would be easier to query things like total revenue per task type down the road. The other pointed out that our invoices rarely needed more than a handful of line items and that a JSON column would save us an entire extra table and the joins that come with it. We went with JSON in the end because it matched how we were actually using invoices in practice, but we noted in our own planning docs that a separate table would be the first thing to reconsider if the application ever needed heavier reporting features.

6. Core Features & Functionality

Role-Based Access Control

CashTrack has two roles, admin and member, and the difference between them is enforced in two places, not just one. On the frontend, the interface itself changes depending on role — an admin sees a client list, a full task board across the whole team, and invoicing tools, while a member sees only the tasks assigned to them and their own payout history. But we learned early on, mostly by trying to break our own app, that hiding a button in React does nothing to stop someone from calling the underlying API endpoint directly. So every PHP endpoint that touches sensitive data checks the logged-in user's role and, for members, filters results down to only the rows tied to their own profile id before anything gets sent back. The frontend restriction is about making the app usable and uncluttered. The backend restriction is what actually keeps the data safe.

Task Management

Tasks support priority levels, free-form tags, an assigned team member, a due date, and a compensation amount. Admins can filter and sort the task board by any of these, which became important once we started testing with more than a handful of sample tasks and realized that a flat, unsorted list gets unusable fast.

The Automated Financial Pipeline

This is the feature the entire project exists to support. Instead of an admin manually typing invoice line items, they select a set of completed tasks for a given client and the system generates a draft invoice from those tasks directly, pulling in the compensation amount already stored on each task. When that invoice is later marked as paid, the system creates a corresponding transaction record and updates the payout entries for whichever team members were tied to the tasks on that invoice. None of this requires a second data entry step, which was the entire point of building the project in the first place.

Team & Client Management

Admins can add new clients and new team members through simple forms. When a new team member is added, the system generates a temporary secure credential for them rather than the admin choosing a password on their behalf, which was a small decision but one that felt like the more responsible way to handle onboarding a new account.

7. The Automated Workflow (Step-by-Step)

The clearest way to explain how the pieces of CashTrack fit together is to walk through what actually happens to a single task from creation to payout.

- 1. An admin adds a new client to the system, entering their contact details once.
- 2. The admin creates a task tied to that client, sets a priority and due date, assigns it to a specific team member, and sets the compensation amount for completing it.
- 3. That team member logs in and sees the task appear on their personal dashboard, scoped only to work assigned to them.
- 4. The team member completes the work and marks the task as complete from their view.
- 5. The admin, reviewing completed tasks for that client, selects the relevant ones and generates a draft invoice, which pulls in the compensation amounts automatically instead of the admin retyping them.
- 6. Once the client pays, the admin marks the invoice as paid.
- 7. The system automatically creates a transaction record for that payment and updates the payout history of every team member whose tasks were included on that invoice.

That last step is really the whole reason this project exists. Nobody has to remember to go update a spreadsheet after the invoice is paid. The connection between "this task got done" and "this person earned this amount of money for it" already exists in the data because of how tasks, invoices, and payouts reference each other, so marking an invoice paid is enough to update everything downstream of it automatically. Watching that update happen the first time we got it

working, with no manual step in between, was honestly one of the more satisfying moments of the whole semester.

8. Challenges Faced & Solutions Implemented

Connecting React to PHP

Neither of us had connected a separately hosted frontend to a backend we wrote ourselves before this project, and the first two weeks were rougher than we expected. Our very first requests from React to PHP failed silently, with no obvious error beyond a vague network failure in the browser console. It took a good amount of searching, and eventually reading through the actual browser error text more carefully instead of skimming it, before we understood we were dealing with a CORS issue rather than a bug in our own logic. Once we knew what we were looking for, adding the correct headers on the PHP side fixed it, but we probably lost close to a full evening to that one problem before we figured out what was actually going wrong.

Database Design

We mentioned this earlier, but it is worth repeating here as a challenge in its own right. Our first schema tried to do too much with too few tables, and it fell apart specifically around the case of one invoice covering multiple tasks. Redesigning the schema partway through the project meant going back and rewriting several PHP endpoints that assumed the old structure, which was frustrating in the moment, but it also taught us more about why normalization matters than any lecture on the topic had up to that point.

Security

Our very first working version of the login system stored passwords as plain text in the profiles table, mostly because we were focused on just getting authentication working at all and treated security as something to add later. Once we sat down to actually add it, we learned about PHP's built-in `password_hash` and `password_verify` functions, which meant we never had to write our own hashing logic and could instead lean on something already tested and trusted by the language itself. Switching over meant we had to reset every test account we had created up to

that point, which was a small annoyance, but it was clearly the right call, and it is not something we would want to explain leaving out of a real report on this project.

State Management

As the app grew, we found ourselves passing the current logged-in user's information down through four or five layers of components just so a deeply nested component could check the user's role. Every time we needed that information in a new place, we had to modify every component in between, even ones that had nothing to do with authentication themselves. Switching to React Context let us make the current user available to any component that needed it directly, without threading it through every layer of the tree, and it made the codebase noticeably easier to work with afterward. We went back and forth on whether this was worth the refactor time given our deadline, but in hindsight it was one of the better decisions we made.

9. Learning Outcomes

Component architecture in React was something we had read about before this project but never really understood until we were forced to build with it under real constraints. Splitting the interface into pieces that each manage their own state stopped being an abstract idea and started being the only way to keep a growing codebase manageable once we had a dozen or more interconnected views.

Building our own REST API also changed how we think about web applications generally. Before this project, an API was something we consumed without much thought about how it worked underneath. Designing our own endpoints, deciding what belongs in a request body versus a URL parameter, and thinking through what each endpoint should and should not allow a given user to do, gave us a much clearer picture of what is actually happening every time an app we use loads new information onto a screen.

Maybe the biggest lesson, though, was learning to accept that an early decision we were attached to, like our first database schema or storing passwords in plain text just to get something working, needed to be undone once we understood the problem better. It felt like a step backward at the time, both times, but each rebuild left us with something more solid than what we had before,

and we walked away from this project with a much better sense of when it is worth stopping to fix something underneath the app instead of building around it.

10. Future Enhancements

- Moving off of XAMPP toward a real hosting setup, such as a frontend host like Netlify paired with a remote MySQL instance, so the app could be reached outside of our own laptops.
- Exporting invoices as PDF files directly from the dashboard instead of only viewing them on screen.
- Email notifications sent to a team member automatically when they are assigned a new task, rather than relying on them checking the dashboard.
- Real-time updates using WebSockets, so a task marked complete by a team member would appear on the admin's screen without needing a page refresh.

None of these made it into the current version, mostly because of time, but they were the natural next steps we kept coming back to in conversation while working on other parts of the app.

11. Conclusion

CashTrack started as a fairly simple idea, that a freelance team's task list and its financial records should be the same underlying data instead of two things someone reconciles by hand, and it grew into a full-stack application with a React frontend, a PHP backend, and a MySQL database, developed and tested locally through XAMPP. Along the way we rebuilt our database schema more than once, spent an embarrassing amount of time chasing down a CORS error, and had to go back and fix a security decision we should have gotten right the first time.

What we walk away with is not just a working dashboard, but a much clearer picture of how the pieces of a real web application actually fit together, because we had to wire nearly all of it ourselves rather than relying on a service that handles the hard parts automatically. This project taught us more about how software actually gets built, mistakes included, than any single tutorial

we followed while working on it, and that is the part of this experience we expect to carry forward past this course.